# Лекция 4

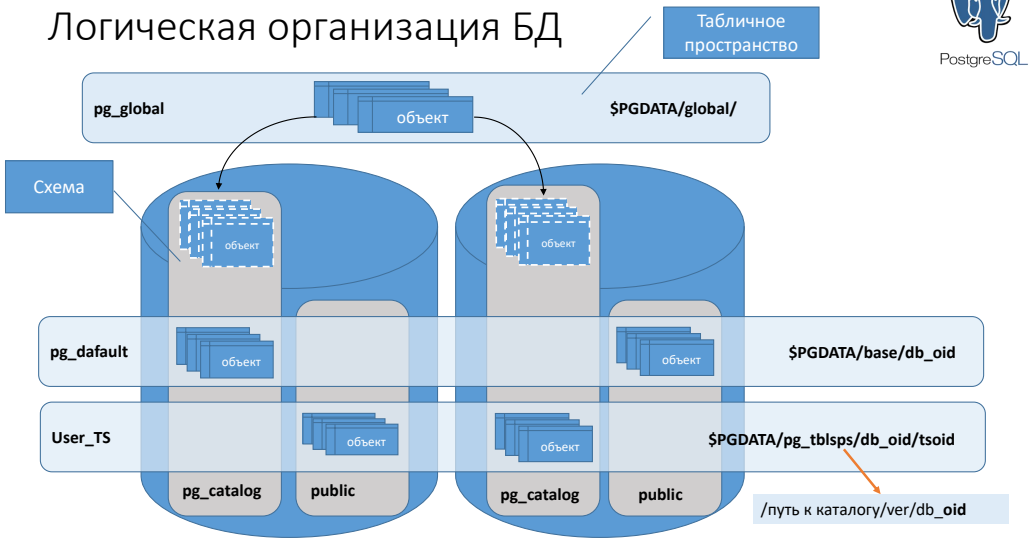
# Создание БД

## Кластер баз данных

- При инициализации кластера БД создается три базы данных:
  - ***postgres*** – используется при подключении по умолчанию пользователем postgres. Не является обязательной, но некоторые утилиты предполагают ее наличие
  - ***template0*** – базовый шаблон
    Не должен изменяться, нужен как минимум в двух ситуациях:
    1. для восстановления БД из резервной копии (**pg_dump**)
    2. при создании новой БД с кодировкой, отличной от указанной при инициализации кластера
  - ***template1*** – шаблон для создания новых БД
    - используется по умолчанию в команде **CREATE DATABASE**
    - может быть удален и создан заново на основе ***template0***
      (необходимо сбросить флаг **pg_database.datistemplate = false**)

```
select *
from pg_catalog.pg_database pd ;
```

## Логическая организация БД



# Создание БД

```
CREATE DATABASE имя
  [ [ WITH ] [ OWNER [=] имя_пользователя ]
      [ TEMPLATE [=] шаблон | DEFAULT]
      [ ENCODING [=] кодировка ]
      [ LC_COLLATE [=] категория_сортировки ]
      [ LC_CTYPE [=] категория_типов_символов ]
      [ TABLESPACE [=] табл_пространство | DEFAULT ]
      [ ALLOW_CONNECTIONS [=] true | false]
      [ CONNECTION LIMIT [=] -1 | предел_подключений ]
      [ IS_TEMPLATE [=] true | false] ]
```

- Необходимо быть суперпользователем или иметь специальное право CREATEDB
- Чтобы шаблон можно было использовать для создания базы, к нему не должно быть активных подключений
- Конфигурационные параметры уровня БД и разрешения уровня БД из шаблона не копируются
- **CREATE DATABASE нельзя выполнять внутри блока транзакции!**

# Примеры создания БД

```
CREATE DATABASE TestDB;

CREATE DATABASE sales
WITH
     OWNER salesapp
     TABLESPACE salesspace; -- табличное пространство д.б. создано заранее!

CREATE DATABASE hr
WITH
   ENCODING = 'UTF8'
   OWNER = hr
   CONNECTION LIMIT = 100   TEMPLATE template0;
--Параметры локали и кодировка должны соответствовать тем, что установлены в
--шаблоне, если только это не template0
```

# Параметры БД

- **IS_TEMPLATE** (**datistemplate**)
  - **true** - пользователи с правом CREATEDB могут клонировать БД
  - **false** - **<u>только</u>** суперпользователь и владелец базы данных могут её клонировать
- **ALLOW_CONNECTIONS** (**datallowconn**)
  - **true** – пользовательские подключения к БД разрешены
  - **false** - новые подключения к этой базе не допустимы
    БД template0 помечена как **datallowconn = false** для избежания любых модификаций
- **CONNECTION LIMIT** (**datconnlimit**)
  - -1 (по умолчанию) - снимает ограничение
  - не распространяется на суперпользователей и фоновые рабочие процессы

```
SELECT datname, datistemplate, datallowconn, datconnlimit
FROM pg_database;
```

# Ошибки

- Нет разрешения на выполнение CREATE DATABASE
- Недостаточно прав в каталоге данных
- Недостаточно места на диске или другие проблемы в файловой системе
- Есть активные подключения к шаблону

# Изменение параметров БД

```
ALTER DATABASE имя [ [ WITH ] параметр [ ... ] ]
ALTER DATABASE имя RENAME TO новое_имя
ALTER DATABASE имя OWNER TO {новый_владелец|CURRENT_USER|SESSION_USER}
ALTER DATABASE имя SET TABLESPACE новое_табл_пространство
```

- Изменение параметров на уровне базы данных:
  ***ALLOW_CONNECTIONS, CONNECTION LIMIT*** и ***IS_TEMPLATE***
- Изменение имени БД.
  Текущую БД переименовывать нельзя (подключитесь к другой БД)
- Изменение владельца БД
- Изменение табличного пространства **по умолчанию** для БД.
  Новое табличное пространство не должно содержать объекты этой БД

```
ALTER DATABASE db RENAME TO appdb;
ALTER DATABASE appdb CONNECTION LIMIT 10;
```

# Удаление базы данных

```
DROP DATABASE имя;
```

- При удалении БД также удаляются все её объекты
  - **Удаление базы данных это необратимая операция!**
- Невозможно выполнить команду DROP DATABASE пока существует хоть одно подключение к заданной базе
  - Нужно подключиться к любой другой (в том числе и template1)
- Нужно быть владельцем БД или суперпользователем

# Схемы

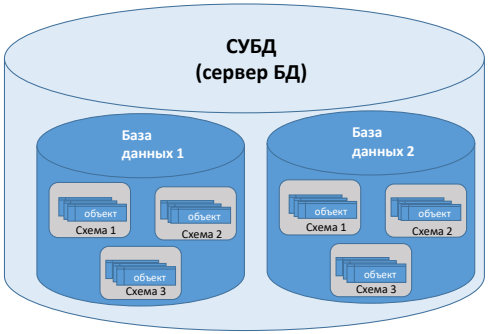
# Именование объектов БД

**Имя_сервера.Имя_БД.Имя_схемы.Имя_объекта**

Пример: Mia-SQL.dbSQL.dbo.”Client”

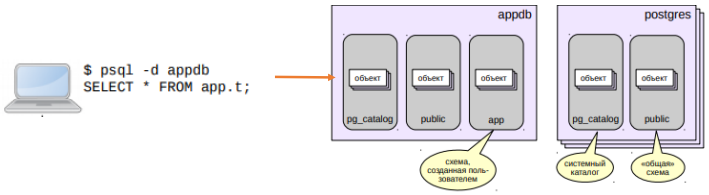
```
SELECT *
FROM dbo."NewTable";
```

- Правила именования объектов
  - Должно начинаться с буквы
  - Может быть длиной до 128 символов*
  - Должно содержать только символы A–Z, a–z, 0–9, _, $ и #
  - Не должно совпадать с именем другого объекта БД
  - Не должно совпадать с зарезервированным словом языка
  - Ограничителем имени является **двойная кавычка** (имена нарушающие правила или чувствительные к регистру)



# Базы данных и схемы

- Кластер баз данных (экземпляр) состоит из баз данных
- База данных содержит различные схемы
- Схемы содержат объекты
  - В каждой БД есть набор стандартных схем: **pg_catalog, public**
  - Пользователь может создавать пользовательские схемы
- Клиент в БД может работать с объектами в любых схемах




# Разрешение имен

- Указание схемы объекта
  - квалифицированное имя объекта явно определяет схему - **схема.имя**
  - имя без квалификатора проверяется в схемах, указанных в **<u>пути поиска</u>**

```
select name, setting
from pg_catalog.pg_settings ps
where name = 'search_path';
```

# Специальные схемы

- Схема **pg_*catalog*** – схема для объектов системного каталога
- Схема ***public*** – схема для размещения объектов по умолчанию
- Схема **pg_*temp***
  - автоматически создается для временных таблиц **<u>в каждом сеансе</u>**
  - по окончании сеанса все объекты временной схемы удаляются

```
select *
from pg_catalog.pg_namespace pn ;
```

# Управление схемами

- Создание схемы

```
CREATE SCHEMA имя_схемы ;
```

- Удаление пустой схемы (не содержащую объектов)

```
DROP SCHEMA имя_схемы;
```

- Удаление схемы со всеми содержащимися в ней объектами

```
DROP SCHEMA имя_схемы CASCADE;
```

# Таблицы

## Создание таблиц

```
CREATE [TEMP][UNLOGGED] TABLE [IF NOT EXISTS]
schema_name.table_name
(column_name datatype [DEFAULT expr][, ...])
```

```
CREATE TABLE "Sales"."CustomerOrders"
                  (OrderID    serial       NOT NULL,
                  OrderDate   date         NOT NULL,
                  CustomerID  int          NOT NULL,
                  Notes       varchar(200) NULL);
```

## Определение столбцов

```
имя_столбца тип_данных
[COLLATE правило_сортировки] [ограничение_столбца [... ]]
```

- **Тип данных** – определяет набор допустимых значений столбца
  - может включать определение массива с этим типом
- **COLLATE** – определяет правило сортировки для символьного столбца
  - Если оно отсутствует, используется правило сортировки по умолчанию, установленное для типа данных столбца
- **Ограничение_столбца** – задают ограничения (проверки), которым должны удовлетворять добавляемые или изменяемые значения

## Типы данных

- Символьные
- Числовые
- Дата и время
- Бинарные
- Логические
- Специальные


Санкт-Петербург политехнический университет Петра Великого С. И. Городецкая

# Генерируемые столбцы

- Поддерживается 3 типа генерируемых столбцов:
  - Вычисляемый столбец на основе выражения
  - Столбец идентификации
  - Столбец SERIAL

https://postgrespro.ru/docs/postgresql/14/ddl-generated-columns



# Вычисляемый столбец на основе выражения

```
GENERATED ALWAYS AS ( генерирующее_выражение ) STORED
```

- Значения в столбце вычисляется с помощью **генерирующего выражения**, которое:
  - может обращаться к другим столбцам данной таблицы
  - **не может** обращаться к другим генерируемым столбцам
  - должно включать только постоянные функции и операторы
- **STORED** – значения вычисляются при записи (добавлении или изменении) и сохраняются на диске
  - виртуальные столбцы (вычисляются при чтении) не реализованы!

```
CREATE TABLE people (
    ...,
    height_cm numeric,
    height_in numeric GENERATED ALWAYS AS (height_cm / 2.54) STORED
);
```



# Столбец идентификации

```
GENERATED {ALWAYS|BY DEFAULT} AS IDENTITY [(параметры_последовательности)]
```

- Автоматически создается **неявная последовательность**, из которой столбец будет автоматически получать значения
  - Столбцу неявно назначается свойство **NOT NULL**
- **ALWAYS**:
  - При вставке (INSERT) пользовательское значение используется, только если указано **OVERRIDING SYSTEM VALUE**
  - При обновлении (UPDATE) любое отличное от **DEFAULT** значение будет отвергнуто
- **BY DEFAULT**:
  - При вставке (INSERT) пользовательское значение имеет приоритет
  - При обновлении (UPDATE) столбец может быть изменён обычным образом
- **параметры_последовательности** позволяют определить свойства последовательности – (START [ WITH ] начало INCREMENT [ BY ] шаг MAXVALUE макс_значение)

```
CREATE TABLE distributors1 (
    did integer GENERATED BY DEFAULT AS
                identity(START WITH 2 INCREMENT by 2),
    name varchar(40)
);
```

# Столбец SERIAL

- В PostgreSQL имеется особый тип для создания столбцов идентификации — **SERIAL**
  - В основе этого типа лежит тип **INTEGER**
  - Столбцу неявно назначается свойство **NOT NULL**
  - Автоматически создается **явная последовательность**
  - Автоматически определяется ограничение **DEFAULT**, которое получает значение с помощью функции `nextval('имя_последовательности')`
- Для каждого поля этого типа создаётся таблица, которая содержит, помимо прочего, текущее число из последовательности
  - При извлечении очередного номера из последовательности текущее число увеличивается
- Получить имя созданной последовательности

```
SELECT pg_get_serial_sequence('table_name','column_name');
```

# Столбцы, содержащие массивы

- Для определения столбца массива необходимо после указания типа данных добавить пару квадратных скобок. Скобки указывают на то, что в этот столбец можно без ограничений вставлять более одного значения описываемого типа данных:
  - **single_array type[]** – Одномерный массив значений
  - **multi_array type[][]** -- Многомерный массив значений

```
CREATE TABLE favorite_books
        (employee_id integer,
         books text[]);
```

```
CREATE TABLE favorite_authors
        (employee_id integer,
         authors_and_titles text[][]);
```

- Преимущества сохранения значений в массиве, по сравнению с сохранением в одной текстовой строке:
  - каждое значение хранится физически отдельно от другого значения в столбце массива;
  - система знает, где начинается и заканчивается каждое значение массива, что позволяет выбирать значения по их индексу, а не вручную анализировать их из длинной текстовой строки

# Изменение и удаление таблиц

# Изменение таблиц

```
ALTER TABLE table_name action;
{RENAME TO  |
ADD COLUMN …| RENAME COLUMN …   | ALTER COLUMN … | DROP COLUMN … [CASCADE]}
```

- Поддерживаемые операции:
  - **Rename To** – переименование таблицы
  - **Add column** – добавление столбца
  - **Rename Column** – переименование столбца
  - **Drop column** – удаление столбца
  - **Alter Column** – изменение характеристик столбца:
    - изменение типа данных
    - переименование столбца
    - задание/изменение значения по умолчанию
    - добавление ограничений целостности*

https://postgrespro.ru/docs/postgresql/14/ddl-alter

# Изменение таблиц

- Переименование таблицы

```
ALTER TABLE "Production"."Products"
RENAME TO "Production"."Product";
```

- Добавление столбца

```
ALTER TABLE "Production"."Product"
ADD COLUMN description text,
ADD COLUMN comments text NOT NULL;
```

- Переименование столбца

```
ALTER TABLE "Production"."Product"
RENAME COLUMN productid TO product_number;
```

- Удаление столбца

```
ALTER TABLE "Production"."Product"
DROP COLUMN description;
```

- Удаление выражения из генерируемого столбца

```
ALTER TABLE payments ALTER COLUMN
income_tax DROP EXPRESSION;;
```

- Изменение столбца

```
ALTER TABLE "Production"."Product"
ALTER COLUMN  productname TYPE varchar(50);
ALTER TABLE "Production"."Product"
ALTER COLUMN unitprice SET DEFAULT 7.77;
```

https://postgrespro.ru/docs/postgresql/14/ddl-alter

# ВАЖНО

- Изменение типа данных столбца может вызвать ошибку, в случае если:
  - Это может привести к усечению существующих данных
  - Преобразование имеющихся данных к указанному типу не поддерживается
  - Существует ссылающийся на данный столбец внешний ключ (FK) в другой таблице
- Удаление столбца, являющегося первичным ключом (PK) не поддерживается при наличие соответствующего внешнего ключа (FK)
  - В случае, если столбец действительно должен быть удален необходимо использовать параметр **CASCADE** - **DROP ... CASCADE**

```
ERROR: cannot drop column product_id of table pk_t because other objects depend on it
 Подробности: constraint fk_t_id_fkey on table fk_t depends on column product_id of table pk_t
 Подсказка: Use DROP ... CASCADE to drop the dependent objects too.
```

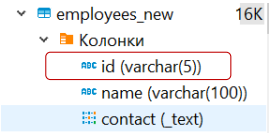
https://postgrespro.ru/docs/postgresql/14/ddl-alter

# Изменение типа данных столбца

- В том случае, когда исходный тип данных столбца изменяется на другой тип данных в пределах одной группы, например, оба типа — числовые, то проблем обычно не возникает
- Если исходный и целевой типы данных относятся к разным группам, тогда потребуется использование фразы **USING** команды **ALTER TABLE**

```
ALTER TABLE seats
 ALTER COLUMN fare_conditions SET DATA TYPE integer
 USING ( CASE fare_conditions
              WHEN 'Economy'  THEN 1
              WHEN 'Business' THEN 2
              ELSE 3
       END );
```

# Пример

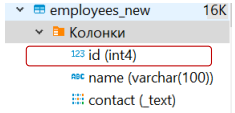



| | id | name | contact |
|---|---|---|---|
| 1 | 1 | Alice John | {(408)-743-9045,(408)-567-7834} |
| 2 | 2 | Kate Joel | {(408)-783-5731} |
| 3 | 3 | James Bush | {(408)-745-8965,(408)-567-78234} |

```
ALTER TABLE public.employees_new
ALTER COLUMN id SET DATA TYPE int;
```

```
SQL Error [42804]: ERROR: column "id" cannot be cast automatically to type integer
 Подсказка: You might need to specify "USING id::integer".
```

```
ALTER TABLE public.employees_new
ALTER COLUMN id  SET DATA type int USING id::int;
```




# Удаление таблиц

```
DROP TABLE [IF EXISTS] table_name [CASCADE| RESTRICT];
```

- Параметр **CASCADE** позволяет удалить таблицу и зависимые от нее объекты
- Параметр **RESTRICT** отклоняет удаление, если от таблицы зависит какой-либо объект
  - Параметр **RESTRICT** используется по умолчанию, если вы не укажете его явно в операторе DROP TABLE
- Для удаления сразу нескольких таблиц необходимо перечислить их в операторе **DROP TABLE** через запятую

https://postgrespro.ru/docs/postgresql/14/ddl-alter

# Создание таблицы на основе выборки

```
CREATE TABLE Table_name
 [ ( Column_name  [, ...] ) ]
AS
SELECT
```

- Количество столбцов, определенных в необязательном списке в круглых скобках, должно соответствовать количеству столбцов выборки
  - если вы указываете необязательный список столбцов в круглых скобках, вы не можете использовать звездочку (*) в операторе select
- Типы данных столбцов должны быть совместимыми
- Если необязательный список столбцов в круглых скобках содержит количество столбцов отличное от возвращаемого оператором select – возвращается ошибка

```
ERROR: CREATE TABLE/AS SELECT has mismatched column count
```

# Создание копии таблицы

```
CREATE TABLE new_table_name
(LIKE source_table_name  { INCLUDING | EXCLUDING }
                         { COMMENTS | COMPRESSION | CONSTRAINTS
                          | DEFAULTS | GENERATED | IDENTITY | INDEXES
                          | STATISTICS | STORAGE | ALL });
```

```
CREATE TABLE student_1
(LIKE student INCLUDING INDEXES INCLUDING COMMENTS);
```

- Создает новую таблицу со структурой родительской таблицы
- **CREATE TABLE LIKE** не копирует какие-либо данные!

# Создание временных таблиц

- Существуют в рамках одной сессии
- Размещаются в схеме **pg_temp**
- Не поддерживают ограничение FOREIGN KEY

```
CREATE TEMPORARY TABLE MyTempTable (cola INT PRIMARY KEY);
GO
INSERT INTO MyTempTable VALUES (1);
```